

Упражнение 5 ЕФП – 2025-2026

Задача 1. Създайте вашата първа програма „hello world“.

1. От меню **File** изберете **New Text File**



2. Запишете следния код във файла:

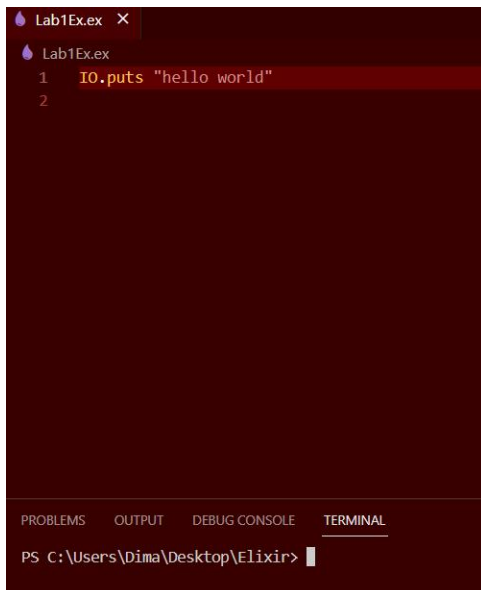
```
IO.puts "hello world"
```

3. Запазете файла с име: **Lab1Ex**. Разширението се добавя автоматично ***.ex**

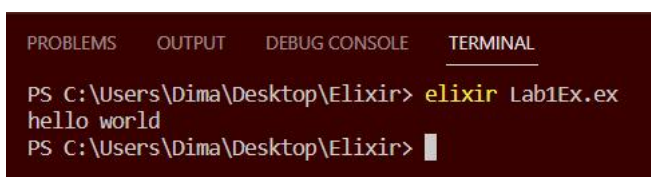
4. От меню **Terminal** изберете:



5. Задайте пътя до вашият файл:



6. За да изпълните файла запишете в терминала: **elixir името_на_файла.ex**



Задача 2. Създайте вашата първа програма, като използвате интерактивния режим за работа. За целта **запишете в терминала - iex.bat**, ако сте на Windows.

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL

PS C:\Users\Dima\Desktop\Elixir> iex.bat
Interactive Elixir (1.13.2) - press Ctrl+C to exit (type h() ENTER for help)
iex(1)> █
```

А изход от този режим **Ctrl+C**

```
PS C:\Users\Dima\Desktop\Elixir> iex.bat
Interactive Elixir (1.13.2) - press Ctrl+C to exit (type h() ENTER for help)
iex(1)> 40+2
42
iex(2)> Terminate batch job (Y/N)? █
```

Работа с колекции

1) **List (Списък).** Списъкът е колекция от подредени стойности, като допуска едновременно елементи от различен тип. Списъките са подходящи за добавяне на елементи в началото на списъка и итерации.

Синтаксис

```
[елемент1, елемент2, ...]
```

Пример:

```
list = [1, 2, 3, :four, "five"]
# Списък с цели числа, атом, и низ
```

2) **Tuple (Кортеж).** Кортежът е колекция с фиксиран по размер. При него има по-бърз достъп по индекса за разлика от списъка. Кортежите не се променят ефективно, при модификация се създава нов.

Синтаксис

```
{елемент1, елемент2, ...}
```

Пример:

```
tuple = {:ok, 200, "OK"}
# Кортеж, често използван за резултати от функции
```

2) **Map (Карта / речник).** Map съхранява двойка ключ–стойност, допустимо е ключовете да са от различен тип.

Синтаксис

```
%{ключ1 => стойност1, ключ2 => стойност2}
```

Пример:

```
person = %{ "name" => "Ivan", "age" => 30 }
# Map с низови ключове
```

Пример:

```
person = %{name: "Maria", age: 25}
# Map с атомни ключове
```

В Elixir операторът `=>` в контекста на дефиницията на Map свързва ключ с неговата стойност и дефакто представлява асоциация **“ключ - стойност”**.

```
# "name" => "Ivan" означава: ключът "name" има стойност "Ivan"
```

Това е синтаксиса при низове. При атоми често се заменя с по-краткия синтаксис с **двоеточие**. т.е. `name: "Maria"` е съкратена форма (syntactic sugar) за `:name => "Maria"`, като двоеточието `:` пред името на ключа дефакто създава атом, който се използва като ключ в Map.

Пример:

```
person = %{name: "Maria", age: 25}
# name: "Maria" е еквивалентно на :name => "Maria"
т.е.
person = %{name: "Maria", age: 25}
# същото като:
person = %{:name => "Maria", :age => 25}
```

4) Keyword List (Ключово/именован списък). Специален списък от двойки [ключ: стойност]. Ключовете са винаги атоми и може да се повтарят.

Синтаксис

```
[key1: value1, key2: value2]
```

Пример:

```
opts = [timeout: 5000, retries: 3]
# Често се ползва за опции към функции
```

Важно! Това е списък, не map — запазва реда и може да има повтарящи се ключове.

5) Range (Интервал). Интервал от числа с фиксирана стъпка (по подразбиране +1). Може да се използва във for и други итерации

Синтаксис

```
start..end
```

Пример:

```
1..5 # Представя числата от 1 до 5
```

Работа със списъци - модул List

1) Списък (List). В Elixir списъкът е подредена колекция от елементи, реализиран като едносвързан списък (linked list) и подходящ за обхождане и рекурсия, а не за директен достъп по индекс. Elixir позволява смесени типове в един списък.

Пример:

```
list = [1, 2, 3, 4]
```

Синтаксис за създаване на списък

```
numbers = [1, 2, 3]
mixed = [1, "text", :atom, true]
nested = [1, [2, 3], 4]
```

Основни операции със списъци:

1) **Достъп до първи елемент (head).** Използва се, когато ни интересува първият елемент.

Пример:

```
hd([10, 20, 30])  
# 10
```

2) **Достъп до останалите елементи (tail).** Кортежите не се променят ефективно, при модификация се създава нов. Използва се при рекурсивно обхождане.

Пример:

```
tl([10, 20, 30])  
# [20, 30]
```

3) **Добавяне на елемент в началото (| оператор).** Най-бързият начин за добавяне в списък.

Пример:

```
list = [2, 3]  
new_list = [1 | list]  
# [1, 2, 3]
```

4) **Конкатенация на списъци (++).**

Пример:

```
[1, 2] ++ [3, 4]  
# [1, 2, 3, 4]
```

По-бавна операция – не е препоръчителна в цикли.

5) **Проверка за “членство” (съществува ли конкретен елемент в списъка).** Полезно при валидации и обхождане на списък.

Пример:

```
3 in [1, 2, 3]  
# true
```

Пример за списък с Enum.each/2:

```
Enum.each([1, 2, 3], fn x ->  
  IO.puts(x)  
end)
```

6) **Когато искаме страничен ефект (например: извеждане).**

Пример за списък с Enum.map/2

```
Enum.map([1, 2, 3], fn x -> x * 2 end)  
# [2, 4, 6]
```

7) **Когато трансформираме данни. Използваме рекурсия със списъци**

Пример:

```
defmodule Sum do  
  def sum([], do: 0)  
  def sum([h | t], do: h + sum(t))  
end
```

8) **Класически подход за обработка на списъци във функционалното програмиране**

Пример с модулет List:

```
List.first([1, 2, 3])  
# 1  
List.last([1, 2, 3])  
# 3
```

Използва се за специализирани операции със списъци.

При използване на генератор (<-) *НЕ се изисква List, а се изисква Enumerable*.

Отново припомниме примера с Pattern matching и използване на генератор (<-)

```
pairs = [{:ok, 1}, {:error, :bad}, {:ok, 2}]

for {:ok, value} <- pairs do
  value
end
# => [1, 2]
```

В този случай имаме **pairs**, което е List, НО for и генераторът (<-) ще работят дори и ако нямашме List.

Какво още може да се постави вдясно от генератора <-?

Всичко, което е Enumerable:

- **Range**

```
for x <- 1..3, do: x
# => [1, 2, 3]
```

- **Map**

```
for {k, v} <- %{a: 1, b: 2}, do: {k, v}
# => [a: 1, b: 2]
```

- **Keyword list (нак е List, но специален)**

```
for {k, v} <- [a: 1, b: 2], do: k
# => [:a, :b]
```

- **Stream**

```
for x <- Stream.cycle([1, 2]), x < 3, do: x
# => [1, 2]
```

Таблица 1. List, Enumerable, генератор и for - кратко пояснение

Понятие	Описание
List	конкретна структура от данни
Enumerable	„нещо, което може да се обхожда“
<-	работи с Enumerable
for	връща List по подразбиране

Важно е да се отбележи, че дори когато входът НЕ е List, резултатът от for по подразбиране винаги е List, освен ако не се добави into:

Пример 1, в който по подразбиране няма into:

```
for x <- 1..3 do
  x * 2
end
# => [2, 4, 6]
Или

for x <- 1..3, do: x * 2
# => [2, 4, 6]
```

При този начин на използване **ВИНАГИ се връща List**, независимо от входа.

Какво прави into:?

into: буквално казва: „Събери резултатите в тази структура, а не в списък“

Синтаксис при добавено into:

```
for ... , into: структура do
```

```
...
end
```

Пример 1, в който има добавено into:

```
for x <- 1..3, into: [] do
  x * 2
end
# => [2, 4, 6]
```

Имаме експлицитно извикване, като into: [] е еквивалент на default.

- генераторът x <- 1..3 обхожда числата 1, 2 и 3
- блокът x * 2 трансформира всеки елемент
- into: [] указва, че резултатът трябва да се натрупа в нов списък, което води до [2, 4, 6]

Без into: резултатът по подразбиране също е **списък**, но с into: може да се **смени структурата** (например Map, MapSet, String).

Пример 2:

Да се създаде for comprehension, който обхожда всеки елемент от keyword списъка, умножава стойността по 10 и формира нов Map, зададен чрез into: %{}.

```
for {k, v} <- [a: 1, b: 2], into: %{} do
  {k, v * 10}
end
# => %{a: 10, b: 20}
```

В този пример използваме into: Map. Входът е keyword list, изходът е Map, а тялото ТРЯБВА да върне {key, value}

Пример 3:

Да се създаде for comprehension, който обхожда списъка и натрупва уникалните елементи в MapSet, като елиминира дубликатите.

```
for x <- [1, 2, 2, 3], into: MapSet.new() do
  x
end
# => #MapSet{1, 2, 3}
```

Разлика между Map и MapSet

- Map пази ключ - стойност
- MapSet пази уникални стойности (без ключове)

Map е асоциативна структура. Това е колекция от двойки ключ-стойност.

Пример:

```
person = %{
  name: "Ivan",
  age: 30
}
```

В Map:

- ключовете са уникални
- стойностите могат да се повтарят
- достъпът е по ключ

```
person[:name]
# => "Ivan"
```

Кога се използва Map?

- когато всяка стойност има ключ
- когато искаме бърз достъп по ключ

MapSet е множество (set). Колекция от уникални стойности, без ключове.

Пример:

```
set = MapSet.new([1, 2, 2, 3])  
# => #MapSet{1, 2, 3}
```

При MapSet:

- няма ключове
- няма дубликати
- редът няма значение

Проверка за членство в MapSet:

```
MapSet.member?(set, 2)  
# => true
```

Кога се използва?

- когато уникалността е важна
- когато трябва да се направи бърза проверка „има ли го конкретния елемент?“

Пример, в който имаме един и същ проблем, но две решения реализирани чрез Map и MapSet

С Map

```
users = %{:1 => "Ivan", 2 => "Maria"}
```

С MapSet

```
online_users = MapSet.new([1, 2, 3])
```

- С Map - се запазват данни за потребители
- С MapSet - се запазва кой потребител е онлайн

Операции, които ги различават

При MapSet има математически операции:

```
a = MapSet.new([1, 2, 3])  
b = MapSet.new([3, 4, 5])  
  
MapSet.union(a, b)  
# => #MapSet{1, 2, 3, 4, 5}
```

```
MapSet.intersection(a, b)  
# => #MapSet{3}
```

Тези операции липсват при Map.

Обобщение

1. Map се използва, когато има структурирани данни и трябва ключове.
2. MapSet се използва, когато ни интересува дали нещо съществува и за връщане на резултат без дубликати

Пример 4:

```
for c <- ?a..?z, c <= ?f, into: "" do  
  <<c>>  
end  
# => "abcdef"
```

В този код се казва, че for comprehension трябва да обхожда всички символи от а до z и да вземе само тези, които са по-малки или равни на f. След това всеки избран символ се преобразува в низ и резултатът се формира като нов низ чрез into: "". Използван е into: със String. Тук: ?a..?z е range от кодове, <<c>> е символ, а into: "" е резултатът е string.

Пример 5:

```
prices = %{:apple: 2, banana: 3}  
  
for {fruit, price} <- prices, into: %{} do  
  {fruit, price * 2}
```

```
end
# => %{apple: 4, banana: 6}
```

В този случай има трансформация на **Map в Map**

ВАЖНО правило!!!

- Типът на резултата зависи от протокола Collectable.
- Това означава: [], %{}, MapSet.new(), "" е ok
- произволна структура не е коректна, ако не е Collectable

Таблица 6. Обобщава случаите на употреба на into

into:	Резултат
няма	List
into: []	List
into: %{}	Map
into: MapSet.new()	MapSet
into: ""	String

Обобщение с едно изречение:

- for винаги произвежда List
- освен ако не му кажеш into: какво да построи

Модул Enum

Модулът Enum в Elixir се използва за обхождане и обработка на колекции (списъци, диапазони, карти и др.). Enum е инструмент за работа с функции като map, filter, reduce, each и др.

1) **Enum-** модул, който предоставя функции за:

- итериране (циклиране);
- филтриране;
- трансформиране;
- агрегация на колекции.

Enum е модул за разлика от List, Map и Range, които са колекции и от Stream, който е модул за lazy (мързелива) обработка на данни.

Основната разлика между модулите Enum и Stream

при Enum:

- обработката става веднага;
- всички елементи се изчисляват наведнъж

Работи с крайни колекции, изчислява резултатите веднага (eager evaluation), т.е. нетърпеливо изпълнение и създава междинни резултати.

Пример:

```
[1, 2, 3, 4]
> Enum.map(&(&1 * 2))
> Enum.filter(&(&1 > 4))
```

Резултат:

- След map междинният резултат е [2, 4, 6, 8]
- След filter междинният резултат е [6, 8]

Тъй като всичко се изчислява веднага имаме по-просто представяне и повече памет при големи данни.

при Stream:

- обработката става само, когато е необходимо;
- елементите се обработват един по един.

Работи с крайни колекции, изчислява резултатите веднага (lazy evaluation). Stream не изчислява нищо веднага, а само описва стъпките.

Пример:

```
[1, 2, 3, 4]
> Stream.map(&(&1 * 2))
> Stream.filter(&(&1 > 4))
> Enum.to_list()
```

Резултат: Нищо не се изпълнява до Enum.to_list/1. Всяка стойност от потока (Stream) се обработва самостоятелно, едновременно с преминаването през всички трансформации, вместо да се създават междинни колекции за всички елементи.

- Stream.map и Stream.filter само описват какво ще се прави, без да създават нов списък веднага
- Когато се извика Enum.to_list(), първо се взема 1, обработва се от map и filter, след това 2, и т.н., всеки елемент се изчислява “на място”
- Така не се създават междинни списъци, което пести памет при големи или безкрайни потоци.

Enum работи с всяка структура, която е Enumerable (**имплементира протокола Enumerable**). Това включва:

- списъци (List)
- map (Map)
- ranges (Range)
- keyword lists
- map sets (MapSet)
- streams (частично)

2) Enum.each/2 – обхождане със странични ефекти**Синтаксис:**

```
Enum.each(колекция, fn елемент -> действие end)
```

Пример:

```
Enum.each([1, 2, 3], fn x ->
  IO.puts(x)
end)
```

Използва се, когато искаме само изпълнение, а не резултат. Връща атом :ok

3) Enum.map/2 – трансформиране на елементи**Синтаксис:**

```
Enum.map(колекция, fn елемент -> нов_елемент end)
```

Пример:

```
squares = Enum.map([1, 2, 3, 4], fn x -> x * x end)
# резултат: [1, 4, 9, 16]
```

Създава нова колекция. Оригиналната остава непроменена.

4) Enum.filter/2 – филтриране по условие**Синтаксис:**

```
Enum.filter(колекция, fn елемент -> условие end)
```

Пример:

```
evens = Enum.filter([1, 2, 3, 4, 5], fn x -> rem(x, 2) == 0 end)
# резултат: [2, 4]
```

Връща само елементите, за които условието е true.

5) Enum.reduce/3 – натрупване на стойност**Синтаксис:**

```
Enum.reduce(колекция, начална_стойност, fn елемент, акумулатор -> нов_акумулатор end)
```

Пример:

```
sum = Enum.reduce([1, 2, 3, 4], 0, fn x, acc -> acc + x end)
# резултат: 10
```

Много мощна функция. Използва се за суми, произведение, конкатенация и др.

6) Enum.sum/1, Enum.max/1, Enum.min/1**Пример:**

```
Enum.sum([1, 2, 3]) # 6
Enum.max([4, 2, 9]) # 9
Enum.min([4, 2, 9]) # 2
```

Готови помощни функции за чести операции

7) Enum.count/1 и Enum.empty?/1**Пример:**

```
Enum.count([1, 2, 3]) # 3
Enum.empty?([]) # true
```

8) Enum.sort/1 и Enum.reverse/1**Пример:**

```
Enum.sort([3, 1, 2]) # [1, 2, 3]
Enum.reverse([1, 2, 3]) # [3, 2, 1]
```

9) Използване с Pipe оператор (>)

```
[1, 2, 3, 4, 5]
> Enum.filter(&(&1 > 2))
> Enum.map(&(&1 * 2))
> Enum.sum()
```

Pipe операторът > подава резултата към следващата функция.

- Enum.filter(&(&1 > 2)) върху [1, 2, 3, 4, 5] дава **[3, 4, 5]** (филтрирай всички стойности по-големи от 2)
- Enum.map(&(&1 * 2)) дава **[6, 8, 10]** (умножи всяка стойност от листа с 2)
- Enum.sum() дава **6 + 8 + 10 = 24** (сумирай всички стойности в листа)

10) Работа с Enum и Map

```
ages = %{ivan: 20, maria: 25}
Enum.each(ages, fn {name, age} ->
  IO.puts("#{name}: #{age}")
end)
```

Задачи за самостоятелна работа

Задача 1. Даден е списък с числа: numbers = [1, 2, 3, 4, 5]. Използвайте Enum.map/2, за да създадете нов списък, в който всяко число е умножено по 3. Краен резултат - [3, 6, 9, 12, 15].

Задача 2. Даден е списък: numbers = [10, 15, 20, 25, 30]. Използвайте Enum.filter/2, за да извлечете само числата, които са по-големи от 20. Краен резултат - [25, 30].

Задача 3. Даден е списък: numbers = [1, 2, 3, 4]. Използвайте Enum.reduce/3, за да намерите сумата на всички елементи. Крайният резултат - 10.

Задача 4. Дадена е Map с цени на стоки:

```
prices = %{
  lamp: 40, chandelier: 120, wall_light: 35
}
```

Използвайте for comprehension, за да създадете нов Map, в който всяка цена е умножена по 2 и се добави ДДС към нея. Използвайте into: %{ } в решението си.

Задача 5. Дадена е Map с потребители и тяхната възраст:

```
ages = %{  
  ivan: 20,  
  maria: 25,  
  petar: 17  
}
```

Използвайте Enum.filter, за да извлечете само хората, които са на 18 или повече години.

Задача 6. Дадена е Map:

```
scores = %{  
  math: 5,  
  physics: 6,  
  chemistry: 4  
}
```

Използвайте Enum.each, за да отпечатате всеки предмет и оценката му във формат:

```
math: 5  
physics: 6  
chemistry: 4
```

Задача 7. Даден е списък с числа: numbers = [1, 2, 2, 3, 3, 4]. Създайте MapSet, който съдържа само уникалните елементи. Краен резултат: #MapSet<[1,2,3,4]>

Задача 8. Дадени са две множества: a = MapSet.new([1, 2, 3]) и b = MapSet.new([3, 4, 5]). Намерете обединението (union) на двете множества.

Задача 9. Даден е MapSet: set = MapSet.new([10, 20, 30]). Проверете дали числото 20 съществува в множеството. Използвайте в решението MapSet.member?

Задача 10. Даден е списък с числа: numbers = [1, 2, 2, 3, 4, 4, 5]. Напишете for comprehension, който

обхожда списъка, оставя само числата по-големи от 2 и събира резултата в MapSet. Използвайте в решението into: MapSet.new(). Краен резултат - #MapSet<[3,4,5]>.

Задача 11. Проектът реализира система за управление на студенти с помощта на езика Elixir. Основната структура се дефинира чрез модул Student и defstruct, който съдържа полетата fname, mname, lname, fnumber и mark. В началото се проверява наличието на текстов файл за съхранение на данни, като при липса се създава празен файл. Менюто на приложението е интерактивно и позволява избор между добавяне, изтриване, показване и обновяване на записи. За четене на данни се използва функцията read_all, която превръща всяка линия от файла в структура Student. Всяка линия се разделя на полета чрез String.split, а оценката се конвертира в float. Записът на данни се извършва чрез функцията write_all, която презаписва файла и добавя всички студенти. Преди добавяне на нов студент се проверява уникалността на fnumber с функцията fn_exists?. При въвеждане на студент се валидират имената с регулярни изрази, а fnumber трябва да е точно 8 цифри. Системата също така проверява дали въведената оценка е валидно число между 2.0 и 6.0. След успешна валидация се създава нова структура Student и се записва във файла. Функцията showAll извежда всички студенти, сортирани по fnumber, като визуализира всички полета. За изтриване на студент се въвежда fnumber, намира се съответният запис и се премахва от списъка. След премахването на записите, обновеният списък се записва отново във файла чрез write_all. Обновяването на студентските данни се извършва чрез меню updateMenu, където се избира полето за промяна. Функцията update намира съответния студент по fnumber и променя избраното поле, като конвертира стойността при нужда. След актуализацията всички записи се записват отново във файла, гарантирайки постоянство на данните. Менюто се извиква рекурсивно след всяка операция, за да се осигури непрекъснатост на интеракцията. Системата комбинира работа с файлове, структурирани данни и валидации за въвеждане, като имитира база данни. В резултат се получава

пълнофункционално приложение за управление на студенти с добавяне, показване, изтриване и обновяване на записи.

```
defmodule Student do
  defstruct fname: "", mname: "", lname: "", fnumber: "", mark: 0.0
  @path "C:\\Users\\vboji\\Desktop\\Elixir\\Struct\\Student_bunch.txt"
  # INIT
  def ensure_file do
    unless File.exists?(@path) do
      File.write!(@path, "")
    end
  end

  # MAIN MENU
  def mainMenu do
    ensure_file()

    IO.puts("\n=====Main Menu=====")
    IO.puts("1. Enter new student")
    IO.puts("2. Delete student")
    IO.puts("3. Show students")
    IO.puts("4. Update student")
    IO.puts("5. Exit")
    case IO.gets("Pick: ") |> String.trim() do
      "1" -> enterStudent()
      "2" -> deleteStudent()
      "3" -> showAll()
      "4" -> updateMenu()
      "5" -> IO.puts("Bye!")
      _ ->
        IO.puts("Invalid choice")
        mainMenu()
    end
  end

  # READ RECORD
  def read_all do
    File.stream!(@path)
    |> Stream.map(&String.trim/1)
    |> Enum.reject(&(&1 == ""))
    |> Enum.map(fn line ->
      [f, m, l, fnumber, mark] = String.split(line, " ")

      %Student{
        fname: f,
        mname: m,
        lname: l,
        fnumber: fnumber,
        mark: String.to_float(mark)
      }
    end)
  end

  # WRITE RECORD
  def write_all(list) do
    File.write!(@path, "")
  end
end
```

```

Enum.each(list, fn s ->
  File.write!(
    @path,
    "#{s.fname} #{s.mname} #{s.lname} #{s.fnomer} #{s.mark}\n",
    [:append]
  )
end)
end

# CHECK FN
def fn_exists?(fnomer) do
  read_all()
  |> Enum.any?(fn s -> s.fnomer == fnomer end)
end

# ADD
def enterStudent do
  fname = IO.gets("First Name: ") |> String.trim()
  mname = IO.gets("Middle Name: ") |> String.trim()
  lname = IO.gets("Last Name: ") |> String.trim()
  fnomer = IO.gets("FN (8 digits): ") |> String.trim()
  mark = IO.gets("Mark (2.00 - 6.00): ") |> String.trim()
  cond do
    !String.match?(fname, ~r/^[a-zA-Z]+$/ ->
      IO.puts("Invalid first name")

    !String.match?(mname, ~r/^[a-zA-Z]+$/ ->
      IO.puts("Invalid middle name")

    !String.match?(lname, ~r/^[a-zA-Z]+$/ ->
      IO.puts("Invalid last name")

    !String.match?(fnomer, ~r/^\d{8}$/ ->
      IO.puts("FN must be exactly 8 digits")

    fn_exists?(fnomer) ->
      IO.puts("FN already exists")

    !String.match?(mark, ~r/^\d+(\.\d{1,2})?$/ ->
      IO.puts("Invalid mark")

  true ->
    {mark, _} = Float.parse(mark)

    if mark < 2.0 or mark > 6.0 do
      IO.puts("Mark must be between 2 and 6")
    else
      s = %Student{
        fname: fname,
        mname: mname,
        lname: lname,
        fnomer: fnomer,
        mark: mark
      }

      File.write!(

```

```

        @path,
        "#{s.fname} #{s.mname} #{s.lname} #{s.fnomer} #{s.mark}\n",
        [:append]
    )
    IO.puts("Student added!")
end
end

mainMenu()
end

# SHOW
def showAll do
    read_all()
    |> Enum.sort_by(& &1.fnomer)
    |> Enum.each(fn s ->
        IO.puts("#{s.fnomer} #{s.fname} #{s.mname} #{s.lname} #{s.mark}")
    end)
    mainMenu()
end

# DELETE
def deleteStudent do
    fnomer = IO.gets("FN to delete: ") |> String.trim()
    list = read_all()
    if Enum.any?(list, fn s -> s.fnomer == fnomer end) do
        new_list = Enum.reject(list, fn s -> s.fnomer == fnomer end)
        write_all(new_list)
        IO.puts("Deleted!")
    else
        IO.puts("Student not found")
    end

    mainMenu()
end

# UPDATE RECORD
def updateMenu do
    IO.puts("\n1. First Name")
    IO.puts("2. Middle Name")
    IO.puts("3. Last Name")
    IO.puts("4. FN")
    IO.puts("5. Mark")
    case IO.gets("Pick: ") |> String.trim() do
        "1" -> update(:fname)
        "2" -> update(:mname)
        "3" -> update(:lname)
        "4" -> update(:fnomer)
        "5" -> update(:mark)
        _ -> IO.puts("Invalid")
    end
    mainMenu()
end

# UPDATE RECORD
def update(field) do

```

```
fnomer = IO.gets("Enter FN: ") |> String.trim()
value = IO.gets("New value: ") |> String.trim()
list = read_all()
updated =
  Enum.map(list, fn
    %Student{fnomer: ^fnomer} = s ->
      case field do
        :mark ->
          {v, _} = Float.parse(value)
          %Student{s | mark: v}
        :fnomer ->
          %Student{s | fnomer: value}
        _ ->
          Map.put(s, field, value)
      end
    s ->
      s
  end)
write_all(updated)
IO.puts("Updated!")
end
end
Student.mainMenu()
```